HRGENIUS · PROJECT GUIDE

Frontend and Backend Code Walkthrough

A step-by-step guide from browser request to database: screens, JavaScript, API routes, controllers, models, authentication, and setup.

Project: HR Management / HRGenius
Guide language: English (technical file names and terms are preserved)
Coverage: source-code walkthrough, file/module map, API flow, and local run guide

1. How to use this guide

This guide explains the current repository source files by module and user flow. “Step by step” means following a browser action through the frontend, API, controller, and MongoDB. It is not a line-by-line commentary or a dump of the source code.

> **Scope:** Frontend HTML/CSS/JavaScript and Backend Express routes, controllers, Mongoose models, middleware, services, and configuration. Binary resume/document samples in `Backend/uploads/` are user files; their contents are not reproduced here.

Project technology at a glance

- **Frontend:** static HTML, CSS, and browser JavaScript; no separate Frontend `package.json` was found.
- **Backend:** Node.js CommonJS, Express 5, MongoDB/Mongoose, JWT, bcryptjs, Multer, and Nodemailer.
- **Data store:** MongoDB. The backend connects using the `MONGO_URI` environment variable.
- **Browser API base:** several frontend scripts use `http://localhost:5000/api`.

Folder map

Path	Purpose
`Frontend/*.html`	Login, dashboard, and HR feature screens and forms.
`Frontend/CSS/`	Shared and feature-specific visual styles.
`Frontend/JS/`	Login, API calls, page behavior, rendering, and theme logic.
`Backend/server.js`	Express app, middleware, API mounts, uploads, and server startup.
`Backend/routes/`	Maps HTTP methods/paths to authorization checks and controllers.
`Backend/controllers/`	Processes requests, calls the database, and returns JSON responses.
`Backend/models/`	Mongoose schemas and relationships for MongoDB documents.
`Backend/middleware/`, `services/`, `config/`	Authentication, permissions, resume uploads, notifications, and email.

2. End-to-end application flow

Step 1 — Screen: The user opens `Frontend/login.html` or a feature page in the browser. HTML defines forms, tables, and modals; CSS styles the layout.

Step 2 — Browser logic: Inline page code or `Frontend/JS/*.js` handles events such as submit, click, and search, then prepares JSON or multipart form data.

Step 3 — API request: The browser calls `http://localhost:5000/api/...` using `fetch()`. After login, protected requests include `Authorization: Bearer <token>`.

Step 4 — Express route: `Backend/server.js` mounts the API prefix to a route module. A route applies authentication, optional role checks, and then invokes a controller.

Step 5 — Controller: The controller uses request data and `req.user` to read/write through Mongoose models, then returns an HTTP status and JSON response.

Step 6 — MongoDB: Mongoose schemas define field types, required values, enums/defaults, and references. Database results flow back through the controller.

Step 7 — UI update: The frontend parses the JSON response and renders tables/cards/stats or displays a success/error message.

Login and a protected request

1. `Frontend/JS/auth.js` sends login details to `POST /api/auth/login`.
2. `authController.js` looks up the user and verifies the password, then returns a token and user details.
3. The browser stores the `token` and user details in `localStorage`.
4. `authMiddleware.js` verifies the Bearer token and puts the decoded identity in `req.user`.
5. `roleMiddleware.js` compares `req.user.role` with the route's allowed roles; a disallowed role receives 403.

Login/token flow sources: `Frontend/JS/auth.js`, `Backend/controllers/authController.js`, `Backend/middleware/authMiddleware.js`, and `Backend/middleware/roleMiddleware.js`.

Resume upload flow

The candidate form sends the resume as a multipart field named `resume`. Candidate routes invoke Multer middleware, which saves the file under `Backend/uploads/`. The configured extensions are PDF/DOC/DOCX and the size limit is 5 MB; uploaded files are served through the `/uploads` static route.

3. Backend startup: `server.js`

1. `dotenv` loads environment values, then the app imports Express, CORS, Mongoose, and path utilities.
2. Route modules are imported for auth, employees, attendance, leaves, payroll, performance, jobs, candidates, interviews, dashboard, departments, audit logs, notifications, and users.
3. Global middleware enables `cors()` and parses JSON and URL-encoded request bodies.
4. `/uploads` serves static files from `Backend/uploads/`.
5. `/api/<resource>` prefixes are mounted on their respective route modules.
6. `/` returns a simple text response; `/api/protected` is an example guarded by auth middleware.
7. Mongoose starts a connection using `MONGO_URI`; the server uses `PORT` or defaults to 5000.

> **When running locally:** Serve the frontend separately with a static server/VS Code Live Server and run the backend in its own terminal. `server.js` does not host the Frontend directory; it only exposes uploads.

Dependencies — `Backend/package.json`

Package	Role
`express`, `cors`	HTTP API and cross-origin browser requests.
`mongoose`	MongoDB connection, schemas, and model operations.
`jsonwebtoken`, `bcryptjs`	JWT authentication and password hashing/comparison.
`multer`, `path`, `fs`	Resume/document uploads and disk storage.
`nodemailer`	Sends password-reset OTP email.
`dotenv`, `nodemon`	Environment configuration and development runner.

4. API surface: route files

Each route file inherits its base path from `server.js`. A route can apply middleware first and then pass the request to a controller function.

Base path	Representative operations
`/api/auth`	`POST /register`, `/login`, `/forgot-password`, `/verify-reset-otp`, `/reset-password`.
`/api/employees`	`POST /`, `GET /`, `GET /:id`, `PUT /:id`, `DELETE /:id`.
`/api/departments`	Create/list/detail/update, `PUT /:id/head`, and delete.
`/api/attendance`	`POST /check-in`, `/check-out`; `GET /`, `/my`, `/employee/:employeeId`.
`/api/leaves`	`POST /create`; `GET /my`, list and employee history; approve/reject.
`/api/payroll`	Create, list, employee payroll, and payment-status update.
`/api/performance`	Create, list, employee review, and review-status update.
`/api/jobs`	Job create/list/detail/update/delete.
`/api/candidates`	Create with resume, list/detail, status update, and delete.
`/api/interviews`	Create/list/detail, status/result update, and delete.
`/api/dashboard`	`GET /stats` for consolidated dashboard statistics.
`/api/notifications`	List/detail, unread count, read/unread, mark all read, and delete.
`/api/users`	`GET/PUT /profile`, `PUT /change-password`.
`/api/audit-logs`	Create and list audit records.

What role checks mean

`authMiddleware` verifies identity; route guards such as `roleMiddleware("admin", "hr")` restrict access by role. Inspect the HTTP method/path and role guard in a route file to understand the intended access pattern. Public login and password-reset endpoints do not use the same guard as protected routes.

Exact route definitions: `Backend/routes/*Routes.js`. Controller implementation: the corresponding resource file under `Backend/controllers/*Controller.js`.

5. Controllers: turning requests into business operations

Controller	Handlers / responsibilities
`authController.js`	Register, login, request a forgot-password OTP, verify OTP, and reset password.
`employeeController.js`	Create, list, retrieve, update, and delete employees.
`departmentController.js`	Create/list/retrieve/update departments, change department head, and delete.
`attendanceController.js`	Resolve the logged-in employee; check in/out; read all, personal, or employee attendance.
`leaveController.js`	Create requests; list all/my/employee leaves; approve or reject.
`payrollController.js`	Create/list payroll, list an employee's payroll, and update payment status.
`performanceController.js`	Create/list reviews, list employee reviews, and update review status.
`jobController.js`	Create, list, retrieve, update, and delete jobs.
`candidateController.js`	Create/list/retrieve candidates, update status, and delete; the upload route links resume file metadata.
`interviewController.js`	Create/list/retrieve interviews; update status/result; delete.
`dashboardController.js`	Assemble dashboard statistics from employees, hiring, leave, attendance, and other HR data.
`notificationController.js`	List/retrieve notifications, count unread items, change read state, and delete.
`userController.js`	Read/update the current profile and change password.
`auditLogController.js`	Create audit records and list logs for admins.

Reading a controller in order

1. Match the exported handler name to the route.
2. See where inputs come from: `req.params`, `req.query`, `req.body`, `req.user`, or `req.file`.
3. Follow Mongoose calls such as `find`, `findOne`, `create`, and `findByIdAndUpdate`.
4. Check early error responses, status codes, and returned JSON: this is the contract visible to the frontend.
5. Trace side effects such as notifications, email, or related records after the main operation.

> **Notification note:** `services/notificationService.js` creates one or multiple notifications. Its catch blocks log errors and return `null` or an empty list, so inspect notification-failure behavior separately from the controller.

6. MongoDB models and data relationships

Each file in `Backend/models/` defines a Mongoose schema/model. Reference fields connect separate collections through ObjectIds.

Model	Core data / relationships
`User`	Name, email, password, role; roles: admin/hr/employee/candidate; password-reset OTP and expiry fields.
`Employee`	Name, email, phone, department, designation, salary, joining date, address.
`Department`	Name, head (Employee reference), description, Active/Inactive status.
`Attendance`	Employee reference, date, check-in/check-out timestamps, Present/Absent/Leave status.
`Leave`	Employee reference, leave type, date range, reason, and Pending/Approved/Rejected status.
`Payroll`	Employee reference, month, basic salary, allowances, deductions, net salary, payment state/date.
`Performance`	Employee reference, review period, goals, achievements, rating, feedback, review status.
`Job`	Title, department, location, job type, salary, description, requirements, status, posting user.
`Candidate`	Name/contact, resume, skills, experience, education, applied Job reference, and application status.
`Interview`	Candidate and Job references, interviewer, date, mode, meeting/location, status, feedback/result.
`Notification`	User reference, title/message/type/priority, read state, action URL, and related-record details.
`AuditLog`	User reference, action, module, description, and IP address.

Relationship sketch

`User` represents a role-based account. An `Employee` belongs to a Department; attendance, leave, payroll, and performance records reference an Employee. A `Job` references its posting User; a Candidate references its applied Job; an Interview references both Candidate and Job. Notifications and AuditLogs reference Users.

Check each model file for the exact `required`, `unique`, `enum`, and `default` constraints. Mongoose constraints do not replace frontend validation.

7. Frontend pages: from HTML to user experience

Page	Purpose and linked logic
`index.html`	Project entry/intro page; shared `style.css`, auth, and theme scripts.
`login.html`	Login form; uses `auth.js` and inline page code for sign-in UI.
`forgot-password.html`	Forgot password → OTP verification → new password; inline API calls to auth reset endpoints.
`dashboard.html`	HR overview/cards/charts; `dashboard.js`, `style.css`, and `dashboard.css`.
`employees.html`	Employee list/form/search/edit/delete; `employees.js`.
`departments.html`	Departments, head assignment, and employee summaries; `departments.js`.
`attendance.html`	Check-in/out and attendance views; `attendance.js`.
`leaves.html`	Leave request/status UI; inline JavaScript calls `/api/leaves`.
`payroll.html`	Payroll rows/forms/status; `payroll.js`.
`performance.html`	Performance records/forms; inline JavaScript calls `/api/performance`.
`jobs.html`	Job posting/list/manage; `jobs.js`.
`candidates.html`	Candidate form, resume, filtering/status; inline JavaScript and `candidates.css`.
`interviews.html`	Interview scheduling/status/results; `interviews.js` and `interviews.css`.
`notifications.html`	Notification feed/read state/filters; `notifications.js` and `notifications.css`.
`settings.html`	Profile/password/settings forms; inline profile, password, and notification fetch logic.

Frontend file groups

- `Frontend/JS/auth.js`: login API, token/user storage, and navigation after login.
- `Frontend/JS/theme.js`: shared theme behavior on page load.
- `attendance.js`, `dashboard.js`, `departments.js`, `employees.js`, `interviews.js`, `jobs.js`, `notifications.js`, `payroll.js`: feature-specific event handlers, API requests, and DOM rendering.
- Leaves, candidates, performance, settings, forgot-password, and some login behavior live in inline `<script>` blocks near the end of their HTML files.

8. How to read frontend JavaScript and CSS

Reading order for a feature JS file

1. At the top, check the API base URL, token/user, and DOM references.
2. Follow the initialization/DOMContentLoaded handler to understand page startup.
3. A `load...` or `fetch...` function retrieves API data; inspect its headers and URL.
4. A render/display function converts JSON into tables, cards, modals, or stats.
5. Form-submit and button handlers send create/update/delete/status actions.
6. Search/filter/sort/pagination functions often update client-side UI state for loaded lists.
7. Success/error helpers show the outcome; logout commonly clears localStorage.

JavaScript file	Feature responsibility
`attendance.js`	Fetch attendance, check in/out, and render records.
`dashboard.js`	Dashboard data, summaries/charts, filtering, and page interactions.
`departments.js`	Department CRUD, head assignment, employee lists, sorting/filtering/search, and modals.
`employees.js`	Role-aware employee CRUD, search, form validation, and feedback.
`interviews.js`	Load candidates/jobs, schedule interviews, and update status/results/feedback.
`jobs.js`	Job CRUD, role controls, search, and list rendering.
`notifications.js`	Feed filters, read/unread actions, delete, pagination, and detail modal.
`payroll.js`	Load payroll data, submit forms, and update payment state.
`auth.js`, `theme.js`	Login/token handling and shared theme setup.

CSS files

- `CSS/style.css`: shared base variables, body, navigation/sidebar, and common links.
- `CSS/dashboard.css`: dashboard layout, sidebar, charts/cards, and responsive rules.
- `CSS/candidates.css`, `departments.css`, `interviews.css`, `notifications.css`: matching page-specific styles.
- `CSS/setting.css`: settings style definitions; check the page's actual stylesheet links—`settings.html` links to the shared `style.css`.

The source hard-codes local API URLs. If the frontend/backend host or port changes, update the relevant script or inline page configuration too.

9. Supporting files and password reset

- `Backend/middleware/authMiddleware.js`: verifies a JWT from the Authorization Bearer header; returns 401 when the token is missing or invalid.
- `Backend/middleware/roleMiddleware.js`: checks whether the logged-in user's role is allowed; returns 403 for a disallowed role.
- `Backend/middleware/uploadMiddleware.js`: Multer disk storage, randomized timestamp filename, PDF/DOC/DOCX extension allowlist, and 5 MB file-size limit.
- `Backend/config/email.js`: Gmail Nodemailer transport; sends an OTP message using `EMAIL_USER`/`EMAIL_PASS`.
- `Backend/services/notificationService.js`: helpers for creating one or many Notification documents.
- `Backend/authMiddleware.js`: a root-level counterpart to the middleware-directory auth file; check route imports when understanding or maintaining the two copies.
- `resetAdminPassword.js`, `resetEmployeePassword.js`: manually invoked password-reset helper scripts, separate from the normal HTTP request flow.
- `testNotification.js`: notification-service test/helper script; it is not part of normal production startup.

Forgot-password flow

1. The forgot-password screen sends an email address to the auth API.
2. The auth controller manages an OTP/expiry and sends an email through `config/email.js`.
3. The user submits the OTP; the verification endpoint checks the OTP and expiry.
4. The reset endpoint saves a new password; the user can then log in normally.

Roles and data access

The model's user roles are `admin`, `hr`, `employee`, and `candidate`. Route permissions are defined file by file, so hiding a frontend menu is not access control: backend route guards form the actual API boundary.

10. Local setup and run steps

1. Install Node.js and prepare a local or reachable MongoDB instance/cluster.
2. Change to the backend directory and install dependencies: `cd Backend`, then `npm install`.
3. Set your local values in `Backend/.env`. Expected names: `PORT`, `MONGO_URI`, `JWT_SECRET`, `EMAIL_USER`, `EMAIL_PASS`. Do not put secret values in source control or a shared PDF.
4. Start the backend with `node server.js`. It defaults to port 5000; check the console for the MongoDB connection result.
5. Open `Frontend/login.html` using VS Code Live Server or another static web server.
6. Use the browser DevTools Network tab to inspect API requests, status codes, responses, and CORS/network errors.

> **Testing status:** `Backend/package.json` currently has a placeholder test command ("no test specified"). This guide does not claim that an automated test suite passed; the app was not launched and runtime-verified during this task.

Typical debugging path

Page issue → HTML element/inline script → linked JS event → Network request/URL/headers → `server.js` mount → route middleware → controller response → model/schema and MongoDB. For upload issues, also check the route, `uploadMiddleware.js`, `req.file`, and `/uploads` serving.

11. Source tree file-by-file index

Source area	Files included
Backend route layer	`authRoutes.js`, `employeeRoutes.js`, `attendanceRoutes.js`, `leaveRoutes.js`, `payrollRoutes.js`, `performanceRoutes.js`, `jobRoutes.js`, `candidateRoutes.js`, `interviewRoutes.js`, `dashboardRoutes.js`, `departmentRoutes.js`, `auditLogRoutes.js`, `notificationRoutes.js`, `userRoutes.js`.
Backend controller layer	`authController.js`, `employeeController.js`, `attendanceController.js`, `leaveController.js`, `payrollController.js`, `performanceController.js`, `jobController.js`, `candidateController.js`, `interviewController.js`, `dashboardController.js`, `departmentController.js`, `auditLogController.js`, `notificationController.js`, `userController.js`.
MongoDB models	`Attendance.js`, `AuditLog.js`, `Candidate.js`, `Department.js`, `Employee.js`, `Interview.js`, `Job.js`, `Leave.js`, `Notification.js`, `Payroll.js`, `Performance.js`, `User.js`.
Frontend pages	`attendance.html`, `candidates.html`, `dashboard.html`, `departments.html`, `employees.html`, `forgot-password.html`, `index.html`, `interviews.html`, `jobs.html`, `leaves.html`, `login.html`, `notifications.html`, `payroll.html`, `performance.html`, `settings.html`.
Frontend scripts/styles	JS: `attendance.js`, `auth.js`, `dashboard.js`, `departments.js`, `employees.js`, `interviews.js`, `jobs.js`, `notifications.js`, `payroll.js`, `theme.js`. CSS: `candidates.css`, `dashboard.css`, `departments.css`, `interviews.css`, `notifications.css`, `setting.css`, `style.css`.

Practical code trace example: leave request

1. Find the form and inline script in `Frontend/leaves.html`.
2. The submit handler sends the form values and token in a `POST /api/leaves/create` request.
3. `Backend/server.js` mounts the request under the `/api/leaves` prefix.
4. `Backend/routes/leaveRoutes.js` applies auth/role checks and invokes the create controller.
5. `Backend/controllers/leaveController.js` processes the request using the Leave model.
6. `Backend/models/Leave.js` defines the employee reference, dates, reason, and status schema.
7. When the response arrives, the frontend refreshes the status/list and shows the result.

12. Suggested learning order

1. **Startup:** `Backend/package.json` → `Backend/server.js`.
2. **Authentication:** `Frontend/login.html` + `Frontend/JS/auth.js` → auth route/controller → User model → auth middleware.
3. **One CRUD feature:** Employees or Departments HTML → JS → route → controller → model.
4. **Role boundaries:** inspect auth and role middleware order in the relevant route file.
5. **Cross-feature behavior:** read the dashboard controller and notification service/controller.
6. **Recruitment:** follow data references from Jobs → Candidates/upload → Interviews.
7. **Operational features:** read Attendance, Leaves, Payroll, Performance, and profile/settings flows.
8. **UI polish:** inspect shared/page-specific CSS and search/filter/modal/theme behavior.

The final mental model

HTML defines screen structure → **CSS** controls appearance → **browser JS** handles interaction/API calls → **Express route** defines URL and permissions → **controller** performs the operation → **Mongoose model** defines the data contract → **MongoDB** persists the data. Trace this chain for one feature, then look for the same pattern in the other modules; a few pages keep the pattern's JavaScript inline in their HTML.

Prepared from the project source tree available in the workspace. No secret values are included. For exact file-level contracts and implementation details, treat the current source files as authoritative.